

```

#include<bits/stdc++.h>
using namespace std;

int main()
{
    int n;
    cout<<"Enter no of vertices:";
    cin>>n;
    vector<vector<int>>adjm(n, vector<int>(n));
    vector<bool>visited(n,false);

    cout<<"Enter adjacency matrix:"<<endl;
    for(auto& i:adjm)
        for(int& j:i)
            cin>>j;
    int node;
    cout<<"Enter node you want to find path";
    cin>>node;

    queue<int>q;
    q.push(node);
    visited[node]=true;
    while(!q.empty())
    {
        int elem=q.front();
        q.pop();

        cout<<elem<<" ";

        for(int i=0; i<n; i++)
        {
            if(adjm[elem][i]==1 && !visited[i])
            {
                //    cout<<"i="<<i<<endl;
                visited[i]=true;
                q.push(i);
            }
        }
    }
}

```

```

#include<bits/stdc++.h>
using namespace std;
void dfs(vector<vector<int>>& adjm, vector<bool>& visited, int n)
{
    visited[n]=true;
    for(int i=0; i<adjm.size(); i++)
    {
        if(adjm[n][i]==1 && !visited[i])
        {
            dfs(adjm, visited, i);
        }
    }
}
int main()
{
    int n;
    cout<<"Enter no of vertices:";
    cin>>n;
    vector<vector<int>>adjm(n, vector<int>(n));
    vector<bool>visited(n,false);
    cout<<"Enter adjacency matrix:"<<endl;
    for(auto& i:adjm)
        for(int& j:i)
            cin>>j;

    dfs(adjm, visited, 0);
    for(bool i:visited)
        if(!i)
        {
            cout<<"The graph is not connected";
            return 0;
        }
    cout<<"The graph is connected";

}

```